

Zaawansowane Technologie Internetowe

Dokumentacja Projektu

System Rezerwacji Terminów Lekarskich

DocBooking

Autor: Artur Saganowski

Technologie: Spring Boot 3.4, Angular 20, PostgreSQL

Data: 30 czerwca 2026

Spis treści

1	Opis funkcjonalności aplikacji	2
1.1	Cel systemu	2
1.2	Funkcjonalności części serwerowej	2
1.3	Funkcjonalności klienta (Angular 20)	2
2	Architektura systemu	3
2.1	Diagram komponentów	3
2.2	Stos technologiczny	3
3	Diagramy UML	4
3.1	Diagram klas – model dziedziny	4
3.2	Diagram przypadków użycia	4
4	Opis kluczowych komponentów	5
4.1	Warstwa bezpieczeństwa – JWT	5
4.2	Programowanie aspektowe – Spring AOP	5
4.3	Komunikacja WebSocket – STOMP	6
5	REST API	7
6	Testy	8
6.1	Testy jednostkowe – backend (JUnit 5 + Mockito)	8
6.2	Testy integracyjne – kontroler REST	8
6.3	Testy frontendu – Angular (Jasmine)	8
7	Informacja uruchomieniowa	9
7.1	Wdrożenie produkcyjne	9
7.2	Wymagania – uruchomienie lokalne	9
7.3	Uruchomienie przez Docker Compose (lokalnie)	9
8	Podręcznik użytkownika	9
8.1	Rejestracja i logowanie	9
8.2	Lista lekarzy i rezerwacja wizyty	10
8.3	Historia wizyt i anulowanie	11
8.4	Panel lekarza	11
8.5	Dane testowe	12
9	JavaDoc – przykłady	12

1 Opis funkcjonalności aplikacji

1.1 Cel systemu

DocBooking to aplikacja webowa klasy klient-serwer umożliwiająca pacjentom rezerwację wizyt lekarskich online. System zapewnia podgląd dostępnych terminów w czasie rzeczywistym dzięki technologii WebSocket, eliminując problem podwójnej rezerwacji tego samego slotu.

1.2 Funkcjonalności części serwerowej

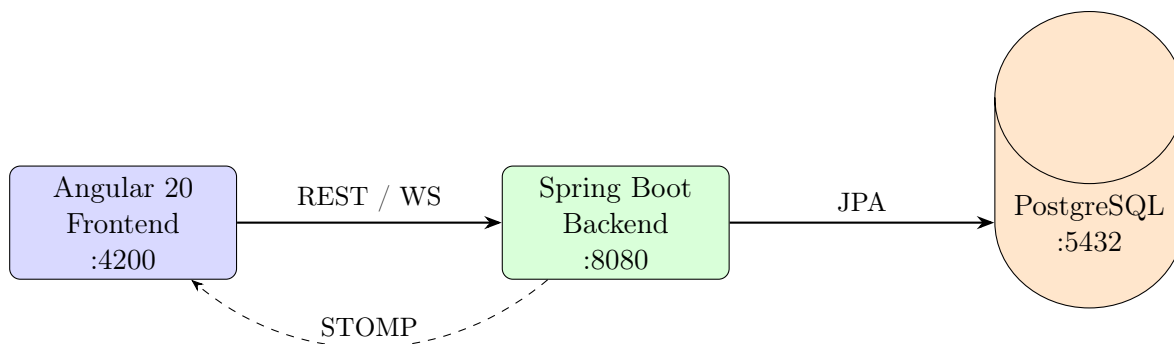
1. **Zarządzanie użytkownikami** – rejestracja pacjentów, logowanie, generowanie i walidacja tokenów JWT.
2. **Zarządzanie lekarzami** – pobieranie listy lekarzy, filtrowanie po specjalizacji.
3. **Zarządzanie slotami** – tworzenie slotów przez lekarza, pobieranie dostępnych terminów, zmiana statusu slotu (`AVAILABLE` / `BEING_RESERVED` / `BOOKED`).
4. **Rezerwacja wizyt** – tworzenie rezerwacji, anulowanie wizyty, historia wizyt pacjenta.
5. **WebSocket** – rozgłaszanie zmian statusu slotów do wszystkich klientów przeglądających terminy tego samego lekarza.
6. **Spring AOP** – automatyczne logowanie każdego wywołania serwisu (czas wykonania, użytkownik, błędy) oraz audit zdarzeń rezerwacji.
7. **Inicjalizacja danych** – seedowanie bazy przy pierwszym uruchomieniu: 3 lekarzy z terminami na 5 dni roboczych.

1.3 Funkcjonalności klienta (Angular 20)

1. **Rejestracja i logowanie** – formularze z walidacją, token JWT w `localStorage`.
2. **Lista lekarzy** – karty Angular Material z danymi lekarza.
3. **Widok terminów** – siatka slotów z live aktualizacją przez WebSocket.
4. **Rezerwacja** – dwuetapowy proces: oznaczenie slotu jako „rezerwowany”, następnie zapis wizyty.
5. **Historia wizyt** – lista wizyt z możliwością anulowania.
6. **Guard i Interceptor** – ochrona tras i automatyczne dołączanie tokenu JWT.

2 Architektura systemu

2.1 Diagram komponentów



Rysunek 1: Diagram komponentów systemu DocBooking

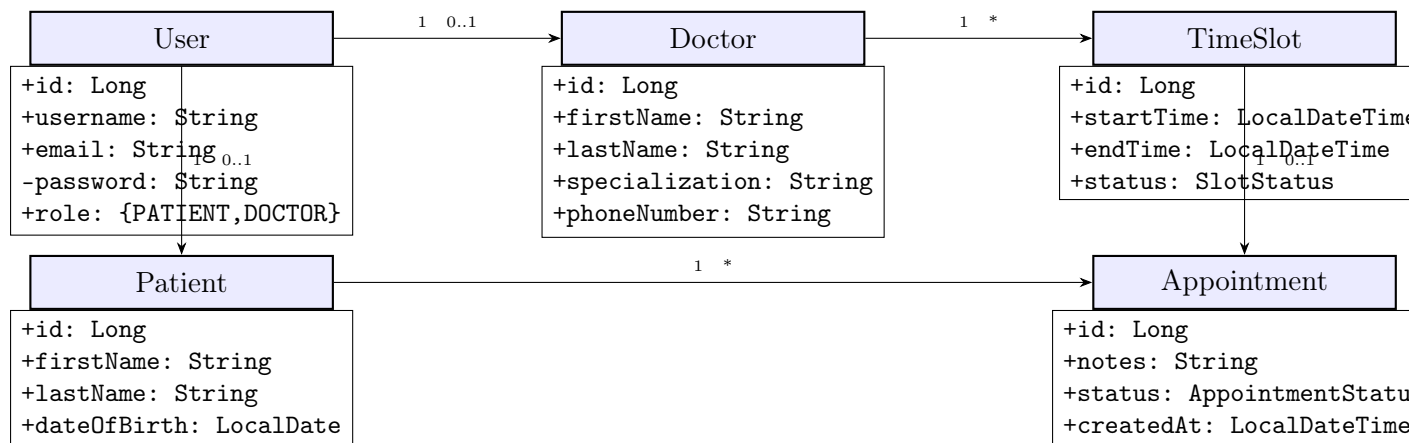
2.2 Stos technologiczny

Warstwa	Technologia	Rola
Frontend	Angular 20	SPA, routing, formularze
HTTP	HttpClient + Interceptor	Żądania REST z nagłówkiem JWT
WebSocket	STOMP.js + SockJS	Subskrypcja live aktualizacji
Backend	Spring Boot 3.4.3	REST API, WebSocket broker
Security	Spring Security + JWT	Autentykacja bezstanowa
ORM	Spring Data JPA	Mapowanie obiektowo-relacyjne
AOP	Spring AOP	Logowanie aspektowe
Baza	PostgreSQL 16	Relacyjna baza danych
Infra	Docker + Compose	Konteneryzacja wszystkich serwisów

Tabela 1: Stos technologiczny aplikacji

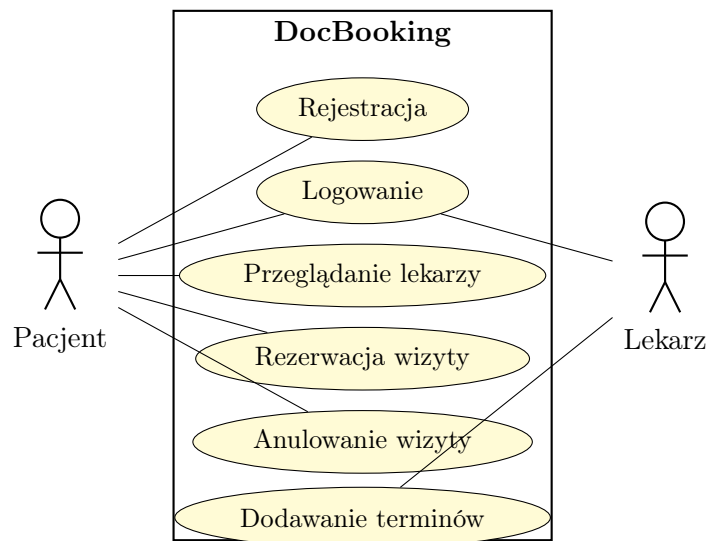
3 Diagramy UML

3.1 Diagram klas – model dziedziny



Rysunek 2: Diagram klas modelu dziedziny (backend)

3.2 Diagram przypadków użycia



Rysunek 3: Diagram przypadków użycia

4 Opis kluczowych komponentów

4.1 Warstwa bezpieczeństwa – JWT

Autentykacja opiera się na tokenach JWT podpisanych kluczem HMAC-SHA256. Po zalogowaniu klient przechowuje token w `localStorage` i dołącza go do każdego żądania przez interceptor.

Listing 1: Generowanie tokenu JWT – `JwtUtil.java`

```
1 public String generateToken(UserDetails userDetails) {  
2     return Jwts.builder()  
3         .subject(userDetails.getUsername())  
4         .issuedAt(new Date())  
5         .expiration(new Date(System.currentTimeMillis() + expiration  
6     ))  
7         .signWith(getSigningKey())  
8         .compact();  
9 }
```

4.2 Programowanie aspektowe – Spring AOP

Aspekt `LoggingAspect` automatycznie loguje każde wywołanie warstwy serwisowej bez modyfikacji kodu biznesowego.

Listing 2: Aspekt logowania – `LoggingAspect.java`

```
1 @Aspect  
2 @Component  
3 public class LoggingAspect {  
4  
5     @Pointcut("execution(* com.zti.doctorreservation.service.*(..))")  
6     public void serviceLayer() {}  
7  
8     @Around("serviceLayer()")  
9     public Object logServiceExecution(ProceedingJoinPoint joinPoint)  
10         throws Throwable {  
11         String method = joinPoint.getSignature().toShortString();  
12         long start = System.currentTimeMillis();  
13         Object result = joinPoint.proceed();  
14         long elapsed = System.currentTimeMillis() - start;  
15         log.info("[{}] {} completed in {}ms",  
16             getCurrentUser(), method, elapsed);  
17         return result;  
18     }  
19  
20     @After("execution(* com.zti.doctorreservation.service  
21         + ".AppointmentService.book(..))")  
22     public void logAppointmentBooked(JoinPoint joinPoint) {  
23         log.info("[AUDIT] Appointment booked by: {}", getCurrentUser());  
24     }  
25 }
```

4.3 Komunikacja WebSocket – STOMP

Listing 3: Rozgłaszanie aktualizacji slotu – TimeSlotService.java

```
1 private void broadcastSlotUpdate(Long doctorId, TimeSlot slot) {  
2     messagingTemplate.convertAndSend(  
3         "/topic/slots/" + doctorId, slot);  
4 }
```

Listing 4: Subskrypcja WebSocket – websocket.ts (Angular)

```
1 subscribeToSlots(doctorId: number): Observable<any> {  
2     const subject = new Subject<any>();  
3     this.client.subscribe(  
4         '/topic/slots/${doctorId}',  
5         (msg: IMessage) => subject.next(JSON.parse(msg.body))  
6     );  
7     return subject.asObservable();  
8 }
```

5 REST API

Metoda	Ścieżka	Opis	Rola
POST	/api/auth/register	Rejestracja	Publiczny
POST	/api/auth/login	Logowanie, zwraca JWT	Publiczny
GET	/api/doctors	Lista lekarzy	AUTH
GET	/api/doctors/{id}	Dane lekarza	AUTH
GET	/api/doctors/{id}/slots	Dostępne terminy	AUTH
POST	/api/doctors/{id}/slots	Dodaj termin	DOCTOR
POST	/api/appointments	Zarezerwuj wizytę	PATIENT
GET	/api/appointments/my	Moje wizyty	PATIENT
PUT	/api/appointments/{id}/cancel	Anuluj wizytę	PATIENT
POST	/api/appointments/slots/{id}/reserve	Oznacz slot	PATIENT

Tabela 2: Endpointy REST API

6 Testy

6.1 Testy jednostkowe – backend (JUnit 5 + Mockito)

Testy jednostkowe weryfikują logikę serwisów w izolacji od bazy danych przy użyciu atrap (Mockito). Przykładowe przypadki testowe dla `AuthService`:

- `register_ShouldCreatePatientAndReturnToken` – poprawna rejestracja zwraca token JWT i tworzy profil pacjenta,
- `register_ShouldThrow_WhenUsernameExists` – duplikat nazwy użytkownika zgłasza `IllegalArgumentException`.

6.2 Testy integracyjne – kontroler REST

Test integracyjny uruchamia pełny kontekst Spring Boot (`@SpringBootTest`) i weryfikuje endpoint `POST /api/auth/register` przez `MockMvc`. Test sprawdza kod odpowiedzi 200 oraz obecność tokenu JWT w odpowiedzi JSON.

6.3 Testy frontendu – Angular (Jasmine)

Test serwisu `AuthService` weryfikuje, że po wywołaniu `login()` token zostaje zapisany w `localStorage` i sygnał `isLoggedIn()` przyjmuje wartość `true`. Używany jest `HttpTestingController` do przechwycenia żądania HTTP.

7 Informacja uruchomieniowa

7.1 Wdrożenie produkcyjne

Aplikacja jest wdrożona i dostępna pod adresem:

<https://perpetual-surprise-production-b801.up.railway.app>

Wdrożenie realizowane jest na platformie Railway z wykorzystaniem kontenerów Docker. Każdy push do gałęzi main wyzwala automatyczny redeploy.

7.2 Wymagania – uruchomienie lokalne

- Docker Engine $\geq$ 24.0 i Docker Compose $\geq$ 2.0
- Wolne porty: 4200 (frontend), 8080 (backend), 5432 (PostgreSQL)

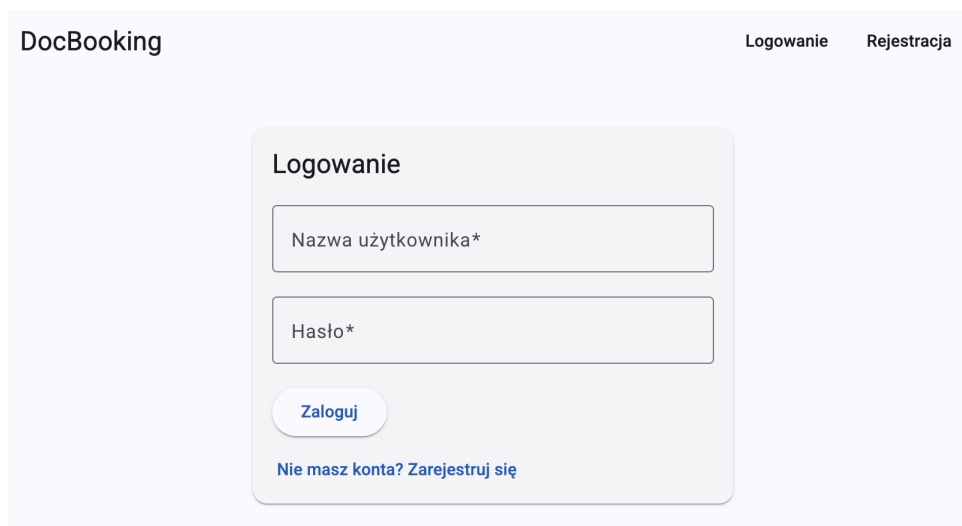
7.3 Uruchomienie przez Docker Compose (lokalnie)

```
git clone https://github.com/REPO/Proj_ZTI.git
cd Proj_ZTI
docker compose up --build
```

8 Podręcznik użytkownika

8.1 Rejestracja i logowanie

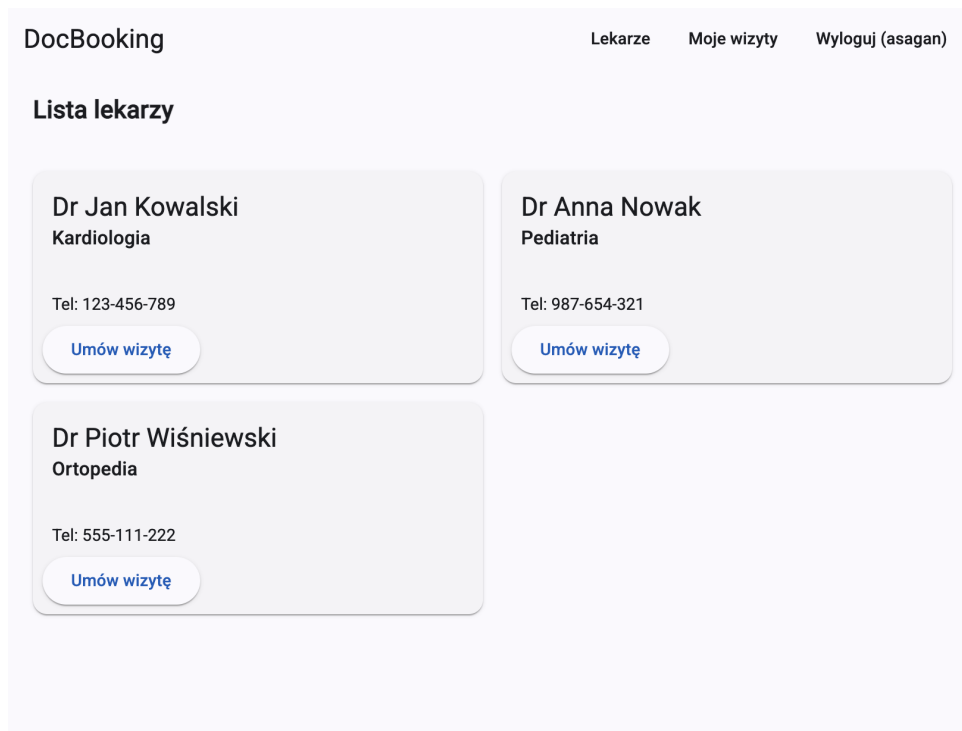
1. Otwórz aplikację, kliknij **Rejestracja**, wypełnij formularz i zatwierdź.
2. Logowanie odbywa się przez **Logowanie** – po poprawnym uwierzytelnieniu system przekierowuje do widoku odpowiedniego dla roli.



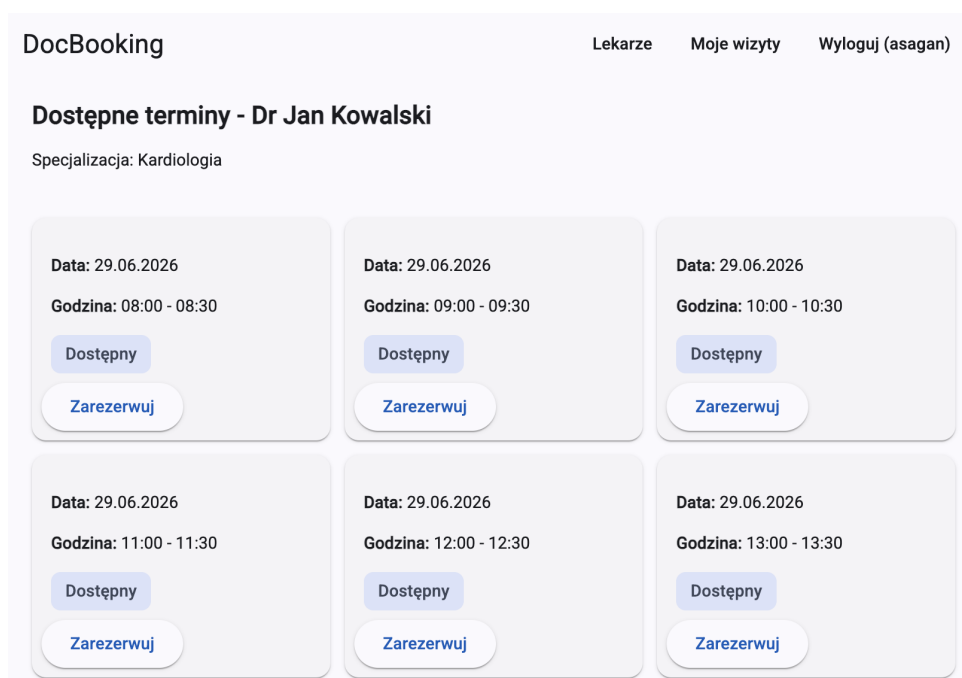
Rysunek 4: Ekran logowania

8.2 Lista lekarzy i rezerwacja wizyty

1. Przejdź do **Lekarze** i wybierz lekarza klikając **Umów wizytę**.
2. Na widoku terminów sloty aktualizują się w czasie rzeczywistym przez WebSocket.
3. Kliknij **Zarezerwuj** – wizyta zostaje zapisana.



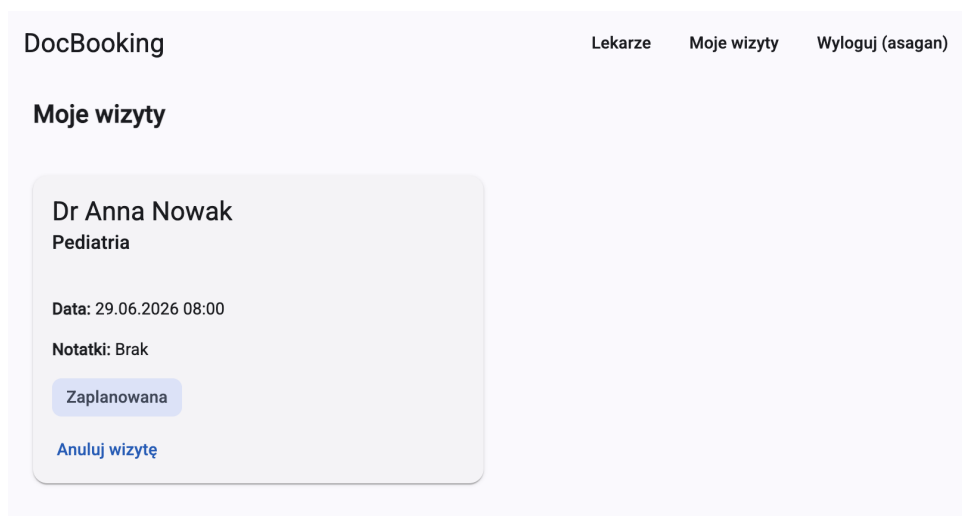
Rysunek 5: Lista lekarzy



Rysunek 6: Widok dostępnych terminów

8.3 Historia wizyt i anulowanie

Pacjent przechodzi do **Moje wizyty** i może anulować zaplanowaną wizytę – slot wraca do puli dostępnych.



DocBooking

Lekarze Moje wizyty Wyloguj (asagan)

Moje wizyty

Dr Anna Nowak
Pediatria

Data: 29.06.2026 08:00

Notatki: Brak

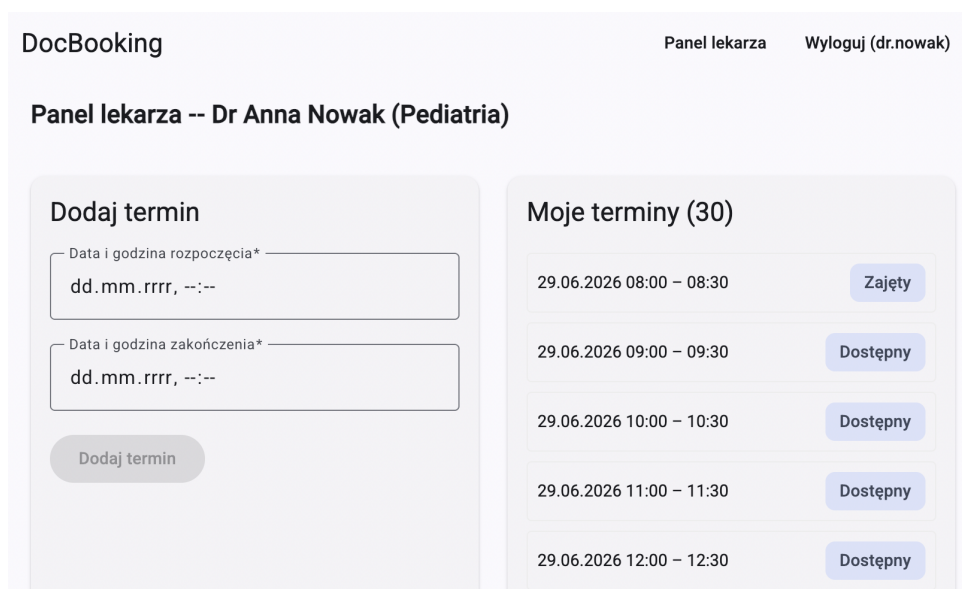
Zaplanowana

[Anuluj wizytę](#)

Rysunek 7: Historia wizyt pacjenta

8.4 Panel lekarza

Lekarz po zalogowaniu widzi zakładkę **Panel lekarza**, gdzie może dodawać nowe terminy wizyt.



DocBooking

Panel lekarza Wyloguj (dr.nowak)

Panel lekarza -- Dr Anna Nowak (Pediatria)

Dodaj termin

Data i godzina rozpoczęcia*
dd.mm.rrrr, --:--

Data i godzina zakończenia*
dd.mm.rrrr, --:--

Dodaj termin

Moje terminy (30)

29.06.2026 08:00 – 08:30	Zajęty
29.06.2026 09:00 – 09:30	Dostępny
29.06.2026 10:00 – 10:30	Dostępny
29.06.2026 11:00 – 11:30	Dostępny
29.06.2026 12:00 – 12:30	Dostępny

Rysunek 8: Panel lekarza – zarządzanie terminami

8.5 Dane testowe

Login	Hasło	Specjalizacja
dr.kowalski	haslo123	Kardiologia
dr.nowak	haslo123	Pediatrica
dr.wisniewski	haslo123	Ortopedia

Tabela 3: Domyślne konta lekarzy (tworzone automatycznie)

9 JavaDoc – przykłady

Listing 5: JavaDoc – AppointmentService.java

```
1  /**
2   * Serwis obslugujacy logik biznesow rezerwacji wizyt.
3   * Po kazdej zmianie statusu slotu rozgasa aktualizacje
4   * przez WebSocket do wszystkich podczonych klientow.
5   */
6  @Service
7  public class AppointmentService {
8
9      /**
10     * Rezerwuje wizyte dla zalogowanego pacjenta.
11     *
12     * @param request obiekt zawierajacy ID slotu i notatki
13     * @return zapisana wizyta
14     * @throws IllegalStateException gdy slot jest niedostepny
15     * @throws IllegalArgumentException gdy slot nie istnieje
16     */
17     public Appointment book(AppointmentRequest request) { ... }
18
19     /**
20     * Anuluje wizyte i zwalnia slot czasowy.
21     *
22     * @param appointmentId ID wizyty do anulowania
23     * @return wizyta ze statusem CANCELLED
24     */
25     public Appointment cancel(Long appointmentId) { ... }
26 }
```